COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 9, 2026
NAME:	Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 SORTING ALGORITHMS PART1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int max_size = 100;

// This function will display the random values
void displayValues(int dataset[])
{
    for (int i = 0; i < max_size; i++)
    {
        std::cout << dataset[i] << " ";
    }
}
```

```cpp
int main()
{
    int dataset[max_size];

    // Generate a different set of random numbers each time
    srand(time(0));

    // Generate 100 random values from 1 to 100
    for (int i = 0; i < max_size; i++)
    {
        dataset[i] = rand() % 100;
    }

    // Display the random values
    std::cout << "Random Values:\n";
    displayValues(dataset);
```

```
Random Values:
7 61 33 69 94 11 60 37 87 3 58 9 95 0 33 82 85 34 25 54 84 1 45 48 39 63 88 67 44 49 78 63 96 37 17 72 77 0 46 33 57 70 12 12 87 18 71 14
 16 33 65 37 39 89 77 36 56 42 92 67 41 91 96 4 54 73 23 97 23 77 36 75 71 79 27 14 8 71 91 18 40 59 23 34 17 1 1 58 17 34 35 42 7 91 19
21 30 76 22 88
```

Observations	This program fills an array of 100 numbers with random values (0-99) using rand(), and srand(time(0)) ensures a different set of numbers appears each run. displayValues then prints them all out, and the console shows they're unordered, as expected before sorting.

Table 7-1. Array of Values for Sort Algorithm Testing

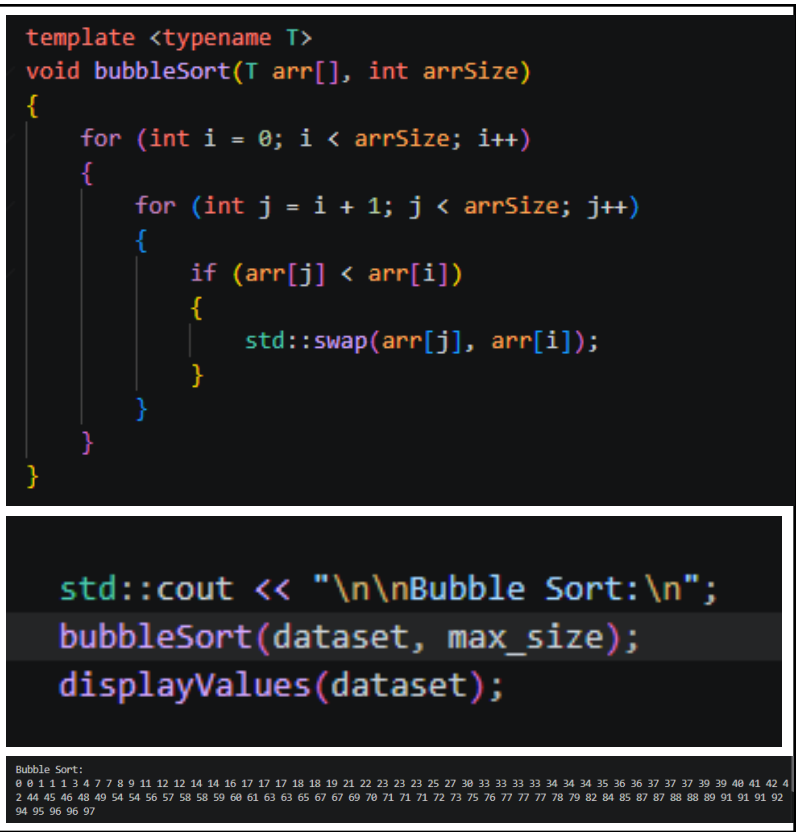
Code + Console Screenshot	
Observations	I observed that bubble sort compares the values in the array and swaps them when they are in the wrong order. It continues comparing and swapping the values until they are arranged from smallest to largest. The code uses two loops to check the elements and std::swap() to change their positions.

Table 7-2 . Bubble Sort Techniques

Code + Console Screenshot	
Observations	The selection sort code works by finding the smallest element in the unsorted part of the array and placing it at the correct position. The routine_smallest() function checks each element and keeps track of the position of the smallest value. Then, selectionSort() uses std::swap() to exchange the smallest value with the current element. I observed that the sorted portion of the array grows from left to right after every iteration until the whole array is sorted. Selection sort always compares the elements in the unsorted part, so its time complexity is **O(n²)**.

Table 7-3. Selection Sort Algorithm

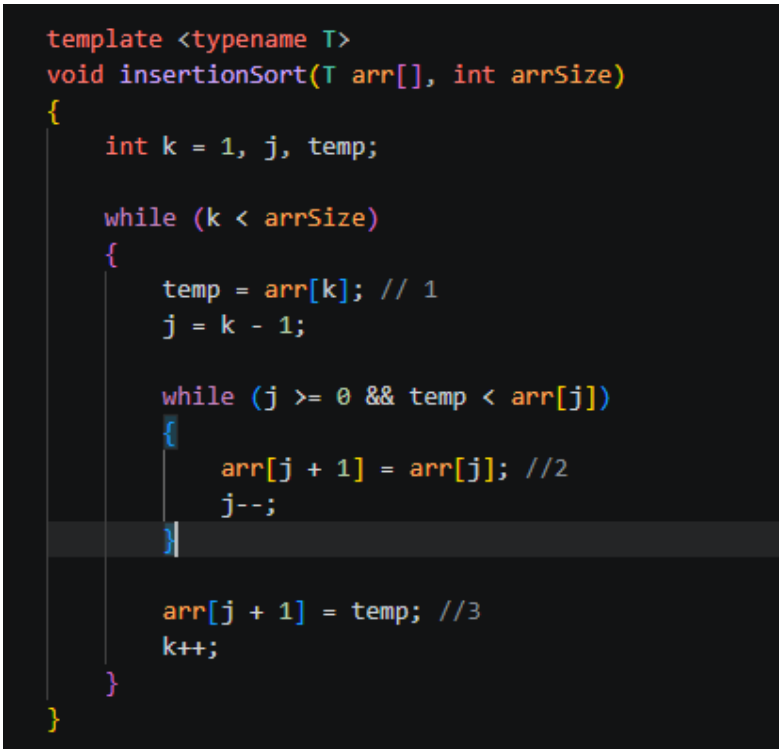
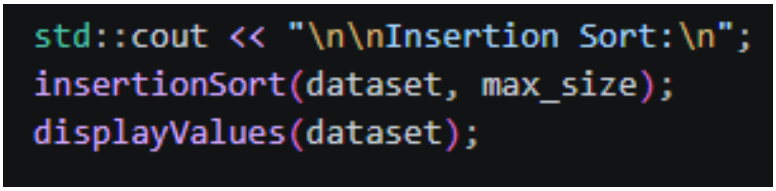
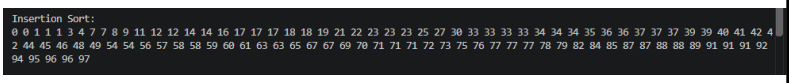
Code + Console Screenshot	  
Observations	I observed that insertion sort takes one value at a time and places it in the correct position among the already sorted values. The code compares the current value with the previous values and moves larger values to the right. It repeats this process until all the elements are arranged from smallest to largest.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithm

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm. (Justify its time complexity)

Deliverables:
- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Pseudocode

```
START

Set max_size = 100
Create an array called votes with 100 elements

Generate 100 random votes from 1 to 5

FOR i = 0 to 99
   votes[i] = random number from 1 to 5
END FOR

Sort the votes using Selection Sort

FOR i = 0 to 98
   Set smallest = i

   FOR j = i + 1 to 99
      IF votes[j] < votes[smallest]
         smallest = j
      END IF
   END FOR

   Swap votes[i] and votes[smallest]
END FOR

Set count1, count2, count3, count4, count5 = 0

FOR i = 0 to 99
   IF votes[i] = 1
      count1++
   ELSE IF votes[i] = 2
      count2++
   ELSE IF votes[i] = 3
      count3++
```

```
   ELSE IF votes[i] = 4
      count4++
   ELSE
      count5++
   END IF
END FOR

Display the sorted votes

Display the vote count of each candidate

Set highest = count1
Set winner = 1

IF count2 > highest
   highest = count2
   winner = 2
END IF

IF count3 > highest
   highest = count3
   winner = 3
END IF

IF count4 > highest
   highest = count4
   winner = 4
END IF

IF count5 > highest
   highest = count5
   winner = 5
END IF

Display the winning candidate and highest vote count

END
```

Main Code

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "selectionsort.h"

const int max_size = 100;

// This function will display the votes
void displayValues(int votes[])
{
    for (int i = 0; i < max_size; i++)
    {
        std::cout << votes[i] << " ";

        if ((i + 1) % 20 == 0)
        {
            std::cout << std::endl;
        }
    }
}

int main()
{
    int votes[max_size];

    // Generate a different set of random numbers each time
    srand(time(0));

    // Generate 100 random votes from 1 to 5
    for (int i = 0; i < max_size; i++)
    {
        votes[i] = rand() % 5 + 1;
    }

    // Sort the votes
    selectionSort(votes, max_size);

    std::cout << "Sorted Array:\n";
    displayValues(votes);

    // Count votes
    int count1 = 0;
    int count2 = 0;
    int count3 = 0;
    int count4 = 0;
    int count5 = 0;
```

```
    for (int i = 0; i < max_size; i++)
    {
        if (votes[i] == 1)
            count1++;
        else if (votes[i] == 2)
            count2++;
        else if (votes[i] == 3)
            count3++;
        else if (votes[i] == 4)
            count4++;
        else if (votes[i] == 5)
            count5++;
    }

    // Display vote count
    std::cout << "\nVote Count:\n";
    std::cout << "Candidate 1: " << count1 << " votes\n";
    std::cout << "Candidate 2: " << count2 << " votes\n";
    std::cout << "Candidate 3: " << count3 << " votes\n";
    std::cout << "Candidate 4: " << count4 << " votes\n";
    std::cout << "Candidate 5: " << count5 << " votes\n";
```

```
    // Find the winner
    int highest = count1;
    int winner = 1;

    if (count2 > highest)
    {
        highest = count2;
        winner = 2;
    }

    if (count3 > highest)
    {
        highest = count3;
        winner = 3;
    }

    if (count4 > highest)
    {
        highest = count4;
        winner = 4;
    }

    if (count5 > highest)
    {
        highest = count5;
        winner = 5;
    }

    std::cout << "\nWinner: Candidate " << winner << " with " << highest << " votes\n";

    return 0;
}
```

Header File

```
#ifndef SELECTIONSORT_H
#define SELECTIONSORT_H
#include <iostream>

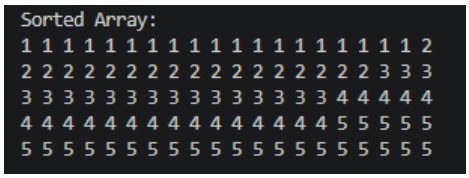
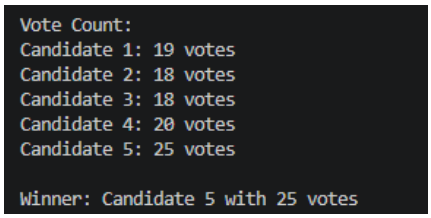
template <typename T>
void selectionSort(T arr[], int arrSize)
{
    for (int i = 0; i < arrSize - 1; i++)
    {
        int smallest = i;

        for (int j = i + 1; j < arrSize; j++)
        {
            if (arr[j] < arr[smallest])
            {
                smallest = j;
            }
        }

        std::swap(arr[i], arr[smallest]);
    }
}

#endif
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
Sorted Array: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Candidate 1: 25 votes Candidate 2: 18 votes Candidate 3: 18 votes Candidate 4: 18 votes Candidate 5: 21 votes	Vote Count: Candidate 1: 25 votes Candidate 2: 18 votes Candidate 3: 18 votes Candidate 4: 18 votes Candidate 5: 21 votes Winner: Candidate 1 with 25 votes
Sorted Array: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Candidate 1: 23 votes Candidate 2: 19 votes Candidate 3: 15 votes Candidate 4: 22 votes Candidate 5: 21 votes	Vote Count: Candidate 1: 23 votes Candidate 2: 19 votes Candidate 3: 15 votes Candidate 4: 22 votes Candidate 5: 21 votes Winner: Candidate 1 with 23 votes

	Vote Count: Candidate 1: 19 votes Candidate 2: 18 votes Candidate 3: 18 votes Candidate 4: 20 votes Candidate 5: 25 votes	

Analysis

I chose Selection Sort because it is simple to understand and easy to use for sorting the 100 votes. Its time complexity is $O(n^2)$ because it repeatedly checks the remaining values to find the smallest one. Although it is not the fastest sorting method, it is suitable for this activity because the number of votes is only 100

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, the developed vote-counting algorithm was effective because it correctly sorted the 100 votes, counted the votes for each candidate, and identified the candidate with the highest number of votes. The results were accurate and easy to check.

Conclusion

In conclusion, I learned how to use Selection Sort to arrange votes and count the votes of each candidate. The algorithm was able to sort the 100 votes and identify the candidate with the highest votes. This activity helped me understand how sorting and counting algorithms can be used to solve real-world problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I confirm that AI did not replace my critical thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.